

Getting Started with HTML5 WebSockets



Ever felt that in the so called two-way Web, updating Web pages in real-time is a pain? Well, the HTML5 specifications took care of that in a big way. Introducing WebSockets—the full power of a UNIX socket, within your Web server.

Seven years ago, AJAX and Web 2.0 were all that was cool. There have been a lot of changes since then. For one thing, HTML5 and CSS3 have pretty much taken the whole world by storm. Apart from making the jobs of novice Web developers just that much easier, it's given the more skilled people many more tools to work with. JavaScript has finally triumphed on the server side, thanks to Google's V8 and Node.js. Cloud computing has finally taken off in a big way and the result has been two very different, yet very similar products putting our Web applications on the cloud—Google App Engine and Windows Azure.

Some of the more low-key advances have been the product of what we call the 'engineer's itch'. We needed it, so we made it. Until now, if you wanted to talk to a Web page and continually update it with real-time data, you had two very inefficient solutions. You either had an HTTP Keep-Alive connection, where the server would keep pushing data to the page. This, depending on how you made it, could even mean that theoretically, the page would never finish loading. Or you could have a script on your page poll the Web server at regular intervals, using... yes, XMLHttpRequests to grab data off the server. Ouch!

WebSockets pretty much makes all of that obsolete.

The idea behind WebSockets

You all know what a socket is. If you don't, shame on you. Go Google it.

All right, you didn't go anywhere, so here's what a socket is, for dummies—it's basically an object where you attach a pipe. Now a pipe is another object; it behaves just like a pipe in your bathroom, except that what's passing through it is not water but data. In the context of the Internet, the pipe is your Internet connection.

So the Web browser opens a socket and connects it to a pipe that goes to the Web server. The Web server has a socket open which is listening for connections—pipes—and once both the server and the client are connected through that pipe, they can talk together, and fetch and push data.

Now sockets are a system feature. If you're writing a Web application, you're probably dealing with HTML, CSS and JavaScript, plus some kind of scripting code at the server end. You shouldn't be dealing with sockets directly. An HTTP connection is an abstraction on top of a socket, and you should be staying within the HTTP connection. But an HTTP connection has limitations: